

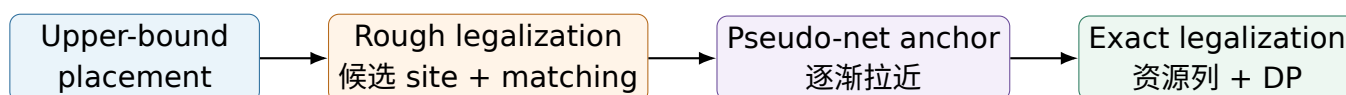
AMF-Placer

第四天学习笔记

Progressive Macro Legalization: 从“可行方向”走到“精确合法位置”

今天只解决一个核心问题：

Macro 不是占一个 site 的普通 cell，而是需要多个相邻 sites、通常还要求落在特定资源列中。AMF-Placer 不在 global placement 早期强行把 macro 一步“钉死”，而是先 rough legalization，再在足够接近合法位置时切换到 exact legalization。



本日覆盖范围

- Section III-D: **Progressive Macro Legalization**
- Algorithm 2: rough legalization 与 exact legalization 的切换
- Min-cost bipartite matching 与 displacement threshold θ_{disp}
- Legalization pseudo net / anchor 的意义
- Intra-column legalization 的动态规划 Eq. (9)
- Tech5 消融实验：为什么 progressive legalization 对大 BRAM macro 尤其重要

目录

1 先回答：为什么 Macro Legalization 比普通 cell 难？	2
2 Progressive 的核心：Rough $\rightarrow$ Exact, 而不是一步到位	3
3 先看论文原始 Algorithm 2	4
4 Rough Legalization 第一步：暂时释放 Macro 的形状约束	5
5 候选 Site: θ_{disp} 控制搜索半径	6
6 核心数学结构：Min-cost Bipartite Matching	7
7 为什么这个 Matching 可以并行？	8
8 Rough Legalization 的输出：不是最终位置，而是 Anchor	9
9 什么时候从 Rough 切换到 Exact？	10
10 Exact Legalization 第一步：先把 Macro 分配到资源列	11
11 论文原始 Eq. (9)：列内 DP	12
12 逐项拆开 Eq. (9)：其实只有“放”和“不放”两个选择	13
12.1 分支 A：让 macro i 在 row j 处结束	13
12.2 分支 B：row j 暂时不用来结束 macro i	13
13 一个教学 Tiny Example：亲手走一遍 DP 的含义	14
14 为什么要保持 Macro 的垂直顺序不变？	15
15 Tech5 消融实验：Progressive Legalization 真的有用吗？	16
16 Day 4 总结：把整条因果链记住	17

1 先回答：为什么 Macro Legalization 比普通 cell 难？

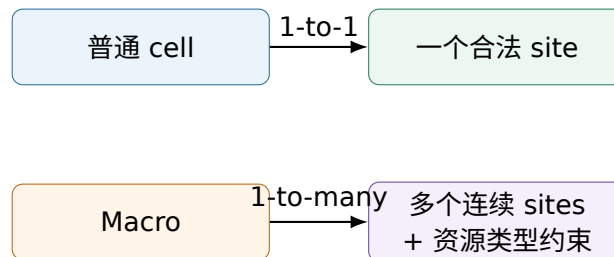
普通 standard cell 的 legalization 通常接近一个 **1-to-1** 问题：一个 instance 找一个可用 site。Macro 不一样，它可能由多个 standard cells 组成，而且这些 cells 之间存在固定的相对位置约束。

例如，假设一个纵向 DSP cascade macro 需要连续占用 3 个 DSP sites，那么它不是：

macro $\rightarrow$ one site,

而是：

macro $\rightarrow$ a block of multiple adjacent legal sites.



此外，DSP、BRAM、LUTRAM 等资源列在 FPGA 上往往比较稀疏。如果 global placement 早期就把一个大型 macro 直接合法化到某个精确位置，它可能被迫产生很大的 displacement，随后 wirelength optimization 也很难再纠正。

最重要的矛盾

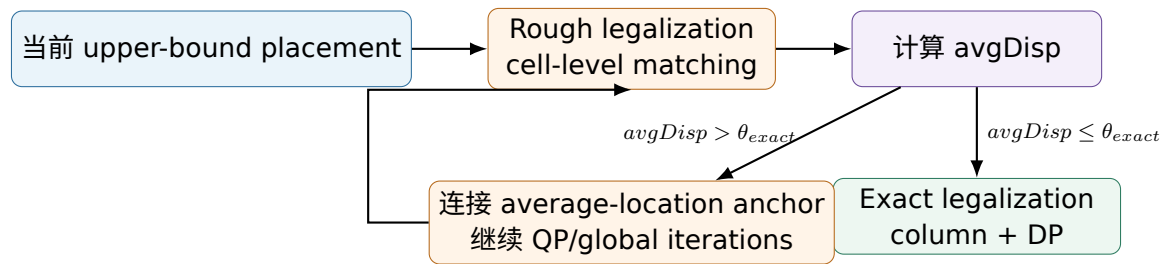
早期： placement 还很不稳定，直接 exact legalization 太“硬”；

后期： 如果一直只做模糊的 rough legalization，macro 又无法真正满足形状和资源约束。

因此 AMF-Placer 采用 **progressive** 策略：先软、后硬。

2 Progressive 的核心: Rough $\rightarrow$ Exact, 而不是一步到位

论文的整体逻辑可以压缩成下面这条状态机:



这里的 $avgDisp$ 可以理解为: rough legalization 后, macro 内各 standard cells 从当前位置到其匹配合法 sites 的平均位移。

为什么用 displacement 决定何时切换?

如果 rough legalization 告诉我们“只需要很小的移动就能找到合法资源”, 说明 macro 已经靠近一个可信的 legal region, 此时再进行 exact legalization, 位移不会太大。反之, 如果候选合法位置还很远, 强行 exact legalization 很可能把 macro 一次拖走很远。

论文还强调: legalization pseudo-net 的权重会随着迭代逐渐增强, 使 macro 平滑靠近合法位置, 而不是突然跳过去。

3 先看论文原始 Algorithm 2

Algorithm 2: Progressive Macro Legalization

Input: upper-bound placement $\mathbf{P}^U = \{(x_i^U, y_i^U)\}$, candidate site number for each cell/macro N_{cand} , initial displacement threshold θ_{disp_I} , threshold increment $\Delta\theta_{disp}$ and displacement threshold to enable exact legalization θ_{exact}

Output: psuedo nets connected to legalization anchors for macros $E_p^{Legal} = \{e_p^{Legal}\}$

```

1 unmappedCells = findUnmappedCellsInMacros();
2 cell2SiteRoughMap =  $\emptyset$ ;
3  $\theta_{disp} = \theta_{disp\_I}$ ;
4 while  $size(unmappedCells) > 0$  do
5   cell2CandidateSiteMap =
     findCandidateSites(unmappedCells,  $\theta_{disp}$ ,  $N_{cand}$ );
6   BPGraphs = createIndependentBipartite-
     Graphs(cell2CandidateSiteMap);
7   parallel foreach  $G$  in  $BPGraphs$  do
8     minCostBipartiteMatching( $G$ );
9   updateRoughLegalization(cell2SiteRoughMap);
10  unmappedCells = findUnmappedCellsInMacros();
11   $\theta_{disp} = \theta_{disp} + \Delta\theta_{disp}$ ;
12 avgDisp = checkAvgDisp(cell2SiteRoughMap);
13 if  $avgDisp > \theta_{exact}$  then
14    $E_p^{Legal} = linkToAvgLoc(cellSiteRoughMap)$ 
15 else
16   macro2ColumnMap = spreadMacrosToCols( $\mathbf{P}^U$ )
17   parallel foreach  $col$  in  $Cols$  do
18     macros = getMacrosIn( $col$ );
19     sortByY( $macros$ );
20     macro2SiteExactMap = DPBasedLegalization( $macro$ ,
         $col$ );
21      $E_p^{Legal} = linkToExactLoc(macro2SiteExactMap)$ ;
```

图 1: 论文 Algorithm 2: Progressive Macro Legalization (直接裁取自原论文第 6 页)。

阅读方法

不要一开始逐行追代码。先把 Algorithm 2 分成两块:

Lines 1-14: Rough legalization; **Lines 15-21: Exact legalization.**

两块之间唯一的“开关”就是 $avgDisp$ 与 θ_{exact} 的比较。

4 Rough Legalization 第一步：暂时释放 Macro 的形状约束

在 rough legalization 阶段，论文不是直接把整个 macro 当作刚体去寻找一个完整合法位置，而是先把 macro 内部的 standard cells 释放出来，分别做 **legalization**。

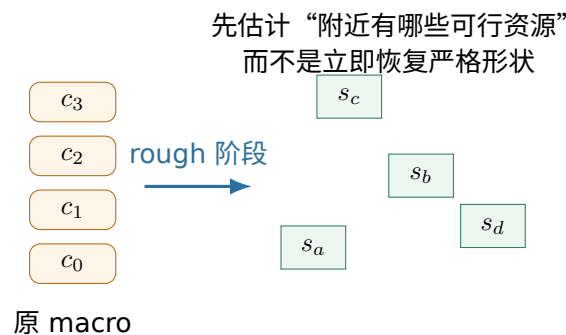
假设一个 macro 由 4 个 cells 组成：

$$M = \{c_0, c_1, c_2, c_3\}.$$

Rough 阶段暂时允许：

$$c_0 \rightarrow s_a, \quad c_1 \rightarrow s_b, \quad c_2 \rightarrow s_c, \quad c_3 \rightarrow s_d,$$

甚至这些 s_a, s_b, s_c, s_d 可能位于不同 columns。



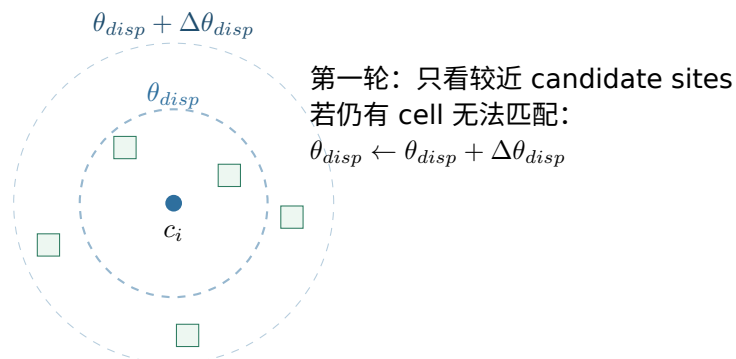
为什么这样做有意义？

它把一个困难的“整个 macro 必须一次匹配连续 sites”的组合约束，暂时降维成多个 cell-to-site 的匹配问题。得到的结果虽然还不是严格合法 macro，但能告诉 placer：这个 **macro** 大致应该往哪里走，哪些资源区域更可行。

5 候选 Site: θ_{disp} 控制搜索半径

对于每个尚未匹配的 cell, AMF-Placer 在给定 displacement threshold θ_{disp} 内寻找最多 N_{cand} 个候选 sites。

可以把它想成一个逐渐扩大的搜索圈：



Algorithm 2 的 while-loop 会不断扩大 θ_{disp} , 直到所有相关 cells 都能找到匹配。

这一步的设计思想

先近后远。这样可优先获得小 displacement 的合法化方向；只有当前局部资源确实不够时，才扩大搜索范围。与一开始就允许任意远的 site 相比，更能避免宏被过早拉到远处。

论文没有给出 $\theta_{disp,I}$ 、 $\Delta\theta_{disp}$ 、 N_{cand} 的具体数值设置，因此本笔记不额外假设这些参数的数值。

6 核心数学结构: Min-cost Bipartite Matching

找到 candidate sites 后, 论文构造一个二分图:

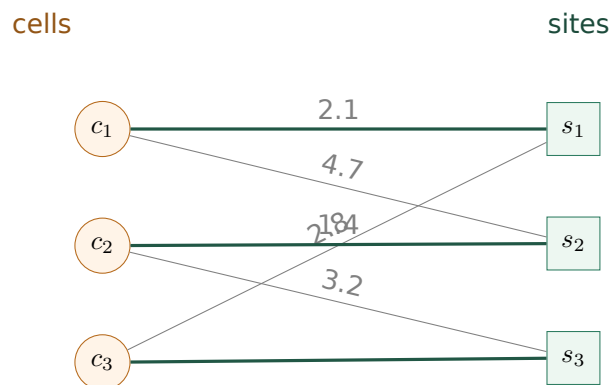
$$G = (C, S, E),$$

其中左侧顶点 C 是需要 legalization 的 standard cells, 右侧顶点 S 是 candidate sites。如果 cell c_i 可以映射到 site s_j , 就添加边 (c_i, s_j) 。

边权不是简单的欧氏距离, 而是论文定义的:

$$\text{cost}(c_i, s_j) = \Delta\text{HPWL}(c_i \rightarrow s_j).$$

然后求 min-cost bipartite matching。



为什么用 ΔHPWL 而不是只用 displacement?

合法化不是单纯“移动最短”。某个 site 虽然几何上更近, 但可能把该 cell 所连 nets 拉得很长。以 HPWL increase 作为 matching cost, 能让 legalization 更直接地保护 global placement 已获得的线长质量。

7 为什么这个 Matching 可以并行？

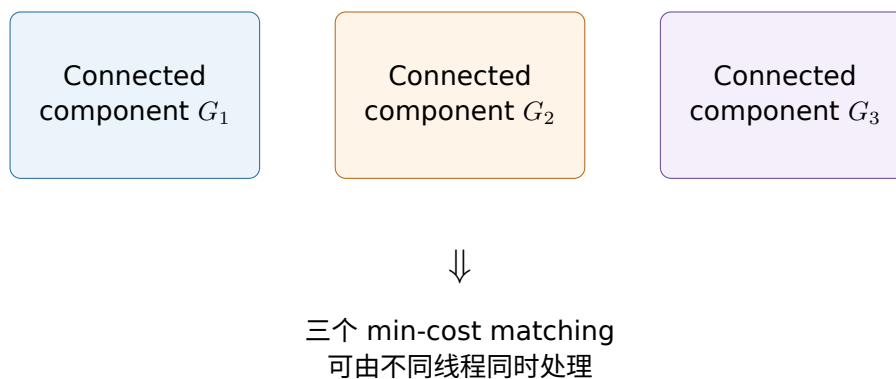
论文指出，当 macros 分散在器件不同区域时，构造出的二分图可能自然分裂成多个互不连接的 connected components：

$$G = G_1 \cup G_2 \cup \cdots \cup G_k, \quad G_p \cap G_q = \emptyset.$$

如果 G_1 与 G_2 没有共享 cell 或 site，那么：

$$\min \text{Cost}(G) = \sum_{t=1}^k \min \text{Cost}(G_t),$$

各子图的 matching 可以独立执行。



算法层面的意义

Progressive legalization 不只是在改善结果质量，它也尽量保持 AMF-Placer 的 parallel-friendly 设计。粗合法化的 bipartite matching、后面的不同资源列 intra-column DP，论文都明确利用了独立性做并行。

8 Rough Legalization 的输出：不是最终位置，而是 Anchor

Matching 完成后，论文对每个 macro 取其内部 standard cells 所匹配 sites 的**平均位置**，并从 macro 到这个平均位置建立 pseudo net。

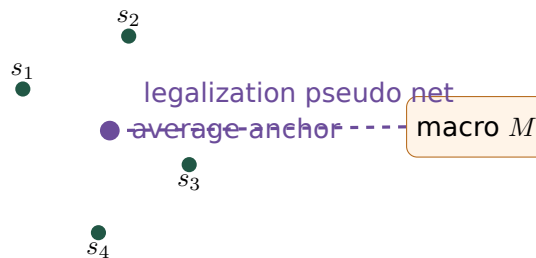
若一个 macro 的 cells 匹配到：

$$(x_1, y_1), (x_2, y_2), \dots, (x_q, y_q),$$

那么教学上可以把论文所说的 average location 写成：

$$\bar{x} = \frac{1}{q} \sum_{k=1}^q x_k, \quad \bar{y} = \frac{1}{q} \sum_{k=1}^q y_k.$$

这里的显式平均公式是为了帮助理解“average location”概念；论文文字给出了取平均位置的操作，但没有单独编号该公式。



为什么不直接把 macro 放到这些 cell 的匹配 sites?

因为 rough 阶段允许 macro 内 cells 暂时分散到不同 columns，它们的 matching 结果只是一个“资源可行方向”。Average anchor 把这些信息压缩成一个连续引导目标，再交回 QP/global iteration，让 macro 平滑靠近，而不是瞬间破坏已有线长结构。

9 什么时候从 Rough 切换到 Exact?

Algorithm 2 的判断条件是：

if $avgDisp > \theta_{exact}$,

则继续 rough legalization, 并把 pseudo net 连到 average location; 否则进入 exact legalization。
也就是：

$$avgDisp \leq \theta_{exact} \implies \text{enable exact legalization.}$$



论文给出的理由是：在 global placement 早期，instance density 高、位置不稳定，DSP/LU-TRAM/BRAM macro 又必须落到稀疏资源区域，有的 macro 甚至跨越几十个 sites。此时直接 legalization 容易产生严重 displacement，并把 placement 陷入坏的 local optimum。

随着 pseudo-net weights 逐步增大，macro 越来越接近 potential legal positions。当 displacement 足够小时，exact legalization 的代价就变得可控。

一个很有价值的理解

θ_{exact} 不只是“几何阈值”，它实际上扮演了一个**阶段切换置信度**：只有当 rough matching 显示“严格合法化不会把 macro 拉太远”时，才把约束从 soft guidance 升级为 hard legalization。

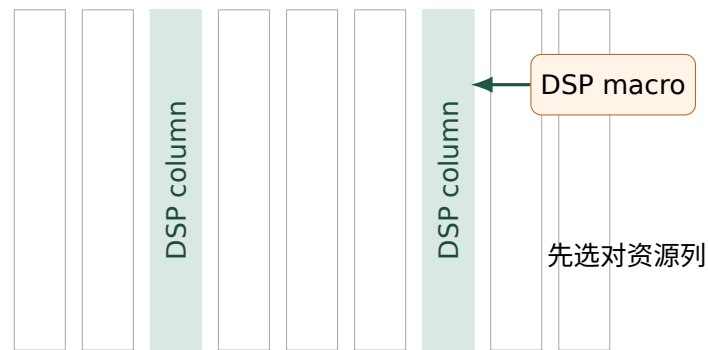
10 Exact Legalization 第一步：先把 Macro 分配到资源列

进入 exact 阶段后，macro 必须恢复真实形状约束。论文首先使用 cell-spreading-based 方法，将 macros 映射到能够提供对应资源类型的 columns。

例如一个 DSP macro：

DSP macro $\Rightarrow$ DSP resource column,

而不能再像 rough 阶段那样让内部 cells 分散在任意位置。



在同一个 resource column 中，仍然需要决定每个 macro 的具体 vertical position。于是问题进一步化简为：

列内合法化问题

给定一个 column，里面有 $N_{macro,col}$ 个 macros 和 $N_{site,col}$ 个可用 rows。每个 macro i 需要连续占用 $row(i)$ 个 rows。怎样安排它们，使 legalization 引入的 HPWL increase 尽量小？

这就是 Eq. (9) 的 dynamic programming。

11 论文原始 Eq. (9): 列内 DP

programming (DP):

$$f(i, j) = \min(f(i-1, j - \text{row}(i)) + \text{HPWL}^+(i, j - \text{row}(i) + 1), f(i, j-1)) \quad (9)$$

where $f(i, j)$ represents the increase of HPWL to legalize the 0- i th macros in 0- j th rows of the column, $\text{row}(i)$ represents the number of adjacent rows which should be occupied by macro i , and HPWL^+ denotes the HPWL increase when macro i is placed from row $j - \text{row}(i) + 1$ to row j , compared to the placement before legalization. Variable i will iterate from 0 to $N_{\text{macro}, \text{col}} - 1$ and Variable j will iterate from 0 to $N_{\text{site}, \text{col}} - 1$. Constraints and initial state settings for Eqn 9 are not shown because of limited space. Since the macros are close to their potential legal positions and the placement tends to be stable at this stage, we assume that the locations of the other instances will remain unchanged during the intra-column legalization of macros. Therefore, the intra-column legalization procedures for different columns can be parallelized (line 17-21 in Algorithm 2).

图 2: 论文 Eq. (9) 及其文字解释 (直接截取自原论文第 7 页)。

论文定义:

$f(i, j) =$ 把第 0 ~ i 个 macros 合法化在第 0 ~ j 行中时的最小 HPWL increase.

$\text{row}(i)$ 表示 macro i 需要占用的连续 rows 数。 $\text{HPWL}^+(i, k)$ 表示将 macro i 放置到以 row k 为起始位置的某段连续 rows 时, 相比 legalization 前增加的 HPWL。

论文明确没有给出的内容

论文由于篇幅限制, 没有写出 Eq. (9) 的完整边界条件和初始状态。因此下面的解释只解析论文给出的 recurrence 本身; 后面的 tiny example 是为了教学构造的示例, 不应误认为是论文原始实验或伪代码。

12 逐项拆开 Eq. (9): 其实只有“放”和“不放”两个选择

论文 recurrence:

$$f(i, j) = \min (f(i-1, j - \text{row}(i)) + HPWL^+(i, j - \text{row}(i) + 1), f(i, j-1))$$

可以把它拆成两个分支。

12.1 分支 A: 让 macro i 在 row j 处结束

若 macro i 需要 $\text{row}(i)$ 行, 并且它的最上端放到 row j , 那么它占用:

$$[j - \text{row}(i) + 1, j].$$

前面的 macros $0 \sim i-1$ 只能使用更下面的 rows, 因此贡献:

$$f(i-1, j - \text{row}(i)).$$

再加上 macro i 这次落位造成的线长代价:

$$HPWL^+(i, j - \text{row}(i) + 1).$$

所以:

$$\text{PlaceCost} = f(i-1, j - \text{row}(i)) + HPWL^+(i, j - \text{row}(i) + 1).$$

12.2 分支 B: row j 暂时不用来结束 macro i

那就把可用范围缩成 $0 \sim j-1$:

$$\text{SkipCost} = f(i, j-1).$$

最后取两者最小:

$$f(i, j) = \min(\text{PlaceCost}, \text{SkipCost}).$$

为什么这个 DP 能保证 macro 不重叠?

“Place”分支一旦把 macro i 放在 $[j - \text{row}(i) + 1, j]$, 前 $i-1$ 个 macros 的子问题只允许使用到 $j - \text{row}(i)$ 。因此两个区域天然不重叠。与此同时, 论文在进入 DP 前已按 y 坐标排序, 并保持 vertical order 不变。

13 一个教学 Tiny Example: 亲手走一遍 DP 的含义

下面不是论文原始数据，只是为了理解 recurrence 构造的小例子。

假设一个 column 有 6 个 rows (编号 0-5)，两个 macros:

$$\text{row}(0) = 2, \quad \text{row}(1) = 3.$$

并且在当前 placement 附近，教学上假设：

Macro	起始 row	占用区间	HPWL ⁺
M_0	0	[0,1]	3
M_0	1	[1,2]	1
M_0	2	[2,3]	4
M_1	2	[2,4]	2
M_1	3	[3,5]	5

如果 M_0 放 [0,1]、 M_1 放 [2,4]，总代价为：

$$3 + 2 = 5.$$

如果 M_0 放 [1,2]，那么为了保持顺序且不重叠， M_1 只能考虑更高的位置 [3,5]，总代价：

$$1 + 5 = 6.$$

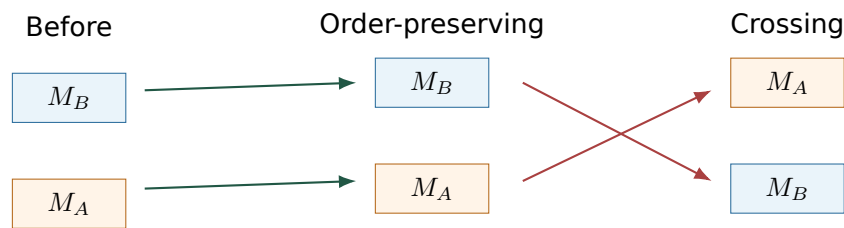
所以这一 toy case 中，DP 会选择第一种组合。

你真正需要抓住的不是数字

DP 不是在“每个 macro 各自找最近 site”，而是在一个 column 内做联合决策：前一个 macro 的落位会改变后一个 macro 的可用空间，因此必须考虑组合代价。

14 为什么要保持 Macro 的垂直顺序不变？

论文在同一 column 中先按当前 y 坐标对 macros 排序，然后在 DP legalization 中保持该顺序。直觉上，这相当于避免宏之间发生大规模“穿越”：



保持顺序让列内 legalization 变成一个一维、可用 DP 求解的问题，也减少了宏之间大规模交换位置的自由度。论文进一步假设：到 exact legalization 阶段，macros 已经靠近潜在合法位置，整体 placement 也趋于稳定，因此在处理某一列时可以把其他 instances 位置视为不变。

不同 columns 之间又是独立的，因此可以：

```
parallel foreach col in Cols: DPBasedLegalization(col).
```

从算法结构看 AMF-Placer

你会发现 AMF-Placer 反复使用同一种思路：

先把全局复杂问题分解成局部独立子问题，再并行求解。

Day 3 的 non-overlap spreading windows 是这样；今天的 independent bipartite subgraphs 和 per-column DP 也是这样。

15 Tech5 消融实验：Progressive Legalization 真的有用吗？

论文把 progressive macro legalization 记为 **Tech5**。消融实验中，“w/o Tech5”表示用 direct macro legalization 替换 progressive strategy。

TABLE II: COMPARISON OF RUNTIME AND HPWL WITH VARIOUS BENCHMARKS AND CONFIGURATION

	Rosetta FaceDetect ³³				SpoonNN ²⁸				OptimSoc ³¹				MiniMap2 ¹⁵			
	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}
proposed	446908	1.000	105	1.054	339764	1.000	129	1.000	1771538	1.000	466	1.000	1275346	1.000	558	1.000
w/o Tech1	439221	1.072	109	1.088	360567	1.061	132	1.023	9034564	5.100	871	1.870	5132073	4.024	1265	2.266
w/o Tech2	465208	1.041	131	1.315	491385	1.446	517	4.007	1841786	1.040	577	1.239	1790566	1.404	1019	1.825
w/o Tech3	487060	1.090	124	1.238	431386	1.270	240	1.857	1825819	1.031	543	1.164	1307249	1.025	626	1.121
w/o Tech4	450199	1.007	100	1.000	351649	1.035	131	1.015	2658863	1.501	525	1.126	1291565	1.013	561	1.005
w/o Tech5	511514	1.145	134	1.344	443765	1.306	138	1.067	1880431	1.061	546	1.172	1430330	1.122	708	1.268
baseline	794201	1.777	234	2.338	1865746	5.491	329	2.549	2231978	1.260	559	1.199	11886747	9.320	3539	6.339
	OpenPiton ³²				Rosetta DigitRecog ³³				MemN2N ²⁹				BLSTM ³⁰			
proposed	1139189	1.000	283	1.035	726929	1.000	254	1.097	801403	1.000	318	1.130	484650	1.000	326	1.241
w/o Tech1	6218009	5.458	350	1.280	768796	1.058	256	1.105	963416	1.202	363	1.287	492194	1.016	285	1.083
w/o Tech2	1159003	1.017	326	1.190	730071	1.004	321	1.389	916176	1.143	411	1.460	496540	1.025	364	1.383
w/o Tech3	1157479	1.016	296	1.079	773957	1.065	272	1.174	862212	1.076	343	1.216	511861	1.056	296	1.126
w/o Tech4	1212149	1.064	274	1.000	728094	1.002	231	1.000	822742	1.027	282	1.000	653010	1.347	307	1.169
w/o Tech5	1142927	1.003	301	1.098	997207	1.372	244	1.053	923670	1.153	353	1.252	722495	1.491	263	1.000
baseline	1432535	1.258	297	1.086	1047885	1.442	298	1.288	1610425	2.010	547	1.942	907383	1.872	325	1.237

TABLE III: ACCELERATION RATIOS WITH PARALLELISM

	Rosetta FaceDetect	SpoonNN	OptimSoc	MiniMap2	OpenPiton	Rosetta DigitRecog	MemN2N	BLSTM
8 threads	2.17x	2.07x	2.50x	2.86x	2.63x	2.22x	2.61x	2.73x

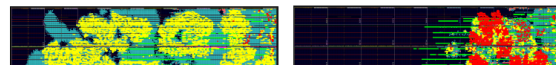


图 3: 论文 Table II 的相关区域（直接裁取自原论文第 8 页）。

从表中可以直接看到，对某些 macro-heavy 设计，去掉 Tech5 的 HPWL 明显变差。例如：

$$\text{SpoonNN: } R_{hpwl} = 1.306, \quad \text{BLSTM: } R_{hpwl} = 1.491.$$

也就是相对 proposed configuration，分别对应约 30.6% 和 49.1% 的 HPWL 增幅。

论文特别指出 SpoonNN 和 BLSTM 含有较多大型 BRAM macros；此类设计中，direct legalization 更容易产生大 displacement，因此 progressive legalization 的收益更加明显。

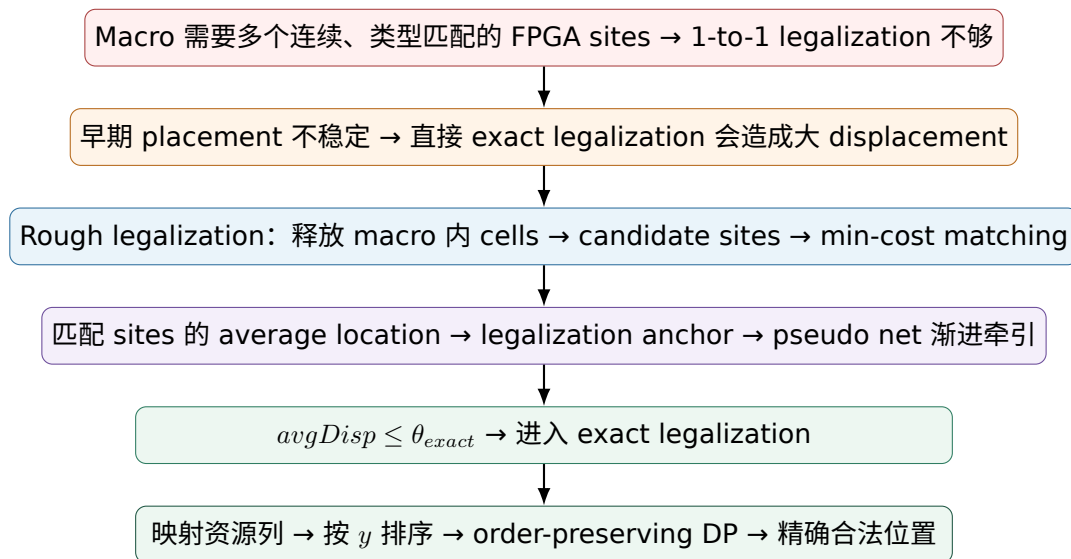
实验与算法逻辑对应起来

大型、稀疏资源 macro 越多：

- ⇒ 精确合法位置越稀缺；
- ⇒ direct legalization 越容易把 macro 一次拖远；
- ⇒ rough anchor + gradual strengthening 越有价值。

16 Day 4 总结：把整条因果链记住

今天最重要的不是背 Algorithm 2，而是把下面的因果链真正理解：



今天应该会回答的 6 个问题

1. 为什么 macro legalization 是 1-to-many?
2. 为什么 early stage 不直接 exact legalization?
3. θ_{disp} 与 $\Delta\theta_{disp}$ 控制什么?
4. bipartite matching 的 edge cost 为什么用 HPWL increase?
5. 为什么 $avgDisp$ 小于阈值后才切换 exact legalization?
6. Eq. (9) 的两个分支分别代表“放 macro i ”和“跳过 row j ”是什么意思?

下一天建议

Day 5 可以进入 **Final Packing + Resource Demand/Supply Adjustment + 实验结果复盘**。重点包括：为什么 pre-packed CLB 仍可继续填充、hash 编码如何避免重复 packing attempts、ripping-up window 如何处理 exception instances，以及 5 个 Tech 的消融结果如何对应前四天的算法机制。